

COMPUTER VISION AND PATTERN RECOGNITION

Project 5

Fine-Tuning LLMs for Keypoint Action Token Imitation Learning in Simulation

Traditional robot learning relies on massive datasets and computationally expensive neural network training (e.g., Behavioral Cloning, Reinforcement Learning). This project explores a paradigm-shifting alternative based on recent literature: controlling a robot without training any models.

By transforming continuous 3D spatial coordinates into discrete text tokens, we can use an off-the-shelf Large Language Model (LLM) to perform robotic tasks. The LLM acts as a sequence-to-sequence predictor driven entirely by a few-shot "In-Context Learning" prompt.

Core paper

Di Palo & Johns, Keypoint Action Tokens Enable In-Context Imitation Learning in Robotics (RSS 2024)

Learning Outcomes

- Identify and extract structured visual representations: Apply concepts of 3D scene understanding and object tracking to extract visual keypoints (3D spatial coordinates) from simulated camera observations.
- Tokenize continuous visual trajectories and physical actions into discrete, text-based sequences.
- Deploy a closed-loop control system integrating real-time visual tracking and LLM inference to solve a physical engineering task.

Core project idea

Select a simulation environment and a compatible manipulation task. Obtain or collect a demonstration dataset from that same environment. Implement a KAT-style observation and action tokenization pipeline, evaluate a pure in-context KAT-style model, and compare it against at least one trained comparison model. Report success rates, generalization behavior, computation cost, and failure cases.

Submission: Please submit a zip file with all code, with detailed readme on how to reproduce. You may choose to submit a GitHub link. A pdf report is also required which will consist of methodology, results table and analysis.

Major Requirements

You can either run a local LLM or use Cloud-only APIs such as GPT, Gemini or Claude. Local setups may include Ollama, LM Studio, vLLM, or Hugging Face Transformers local inference.

You must perform few-shot fine-tuning on each selected task. Minimum settings: 5, 10 and 20 demonstrations.

You must also use at least two different tasks from two different simulation environments. For each task, you must report how performance changes as the number of demonstrations increases.

Examples of acceptable choices:



- **ALOHA (ACT):** Transfer Cube, Bimanual Insertion.
- **PushT (Diffusion Policy):** PushT block-pushing.
- **VQ-BeT environments:** Kitchen, PushT, UR33 or another VQ-BeT-supported task

Note: Using PushT from both Diffusion Policy and VQ-BeT does not count as two different environments. A valid pair would be, for example, ALOHA Transfer Cube + PushT, or ALOHA Insertion + VQ-BeT Kitchen.

Background

Aspect	What KAT does in the paper
Setting	Real robot experiments on a tabletop manipulation setup.
Observation	A single top-down RGB-D image is captured at the beginning of each episode.
Visual tokens	DINO-based dense descriptors are used to extract a set of 3D visual keypoints, which are serialized as text tokens.
Action tokens	Each end-effector pose is represented as a triplet of 3D points in world frame, plus a gripper-open / gripper-closed bit.
Model	GPT-4 Turbo is the main model; the paper also compares GPT-3 Turbo 16k and GPT-2.
Learning mode	In-context imitation only. No additional training or weight updates after demonstrations.
Data regime	The main quantitative table uses 10 demonstrations per task; ablations use 5 / 10 / 20 / 40 demos.
Summary	Records one RGB-D observation at the beginning of an episode. Extracts DINO-ViT dense visual descriptors. Converts selected visual descriptors into 3D keypoints. Converts end-effector poses into action tokens using a triplet of 3D virtual points plus gripper state. Places several demonstration input-output pairs into an LLM prompt. Uses the LLM to generate action tokens autoregressively. Converts predicted action tokens back into executable robot poses.

Project Requirements and Tasks

The project should preserve the spirit of the method: Represent observations and actions as compact text tokens, use a language model as a sequence predictor, and evaluate whether few demonstrations are enough to produce useful actions.

But you may adapt the exact tokenization to the chosen simulator. For example, PushT has 2D planar actions, while ALOHA has 14-dimensional bimanual joint actions; neither exactly matches the paper's Sawyer end-effector SE(3) action format.

Follow these steps:

Step 1: Environment setup and task validation

1. Demonstrate that the environment launches, resets reproducibly, renders observations, and exposes the task success condition.
2. Save one short video showing the robot state, object randomization, and any camera viewpoints used for learning.

Step 2: Dataset acquisition or collection

1. Official simulator demos are acceptable if they are clearly documented and come from the



same simulator and task used for evaluation.

2. Custom-collected demos are also acceptable, but document the collection interface, action schema, and train / test split policy.

Step 3: KAT tokenization pipeline

1. Observation side: Convert RGB-D (or equivalent visual data) into 3D keypoints expressed in a consistent frame. Use DINO/DINOv2/ViT features or simulator-provided object positions to extract keypoints. If depth or object-state is available, convert keypoints to 3D coordinates. For environments without depth, extract 2D image keypoints and represent them as normalized image coordinates.

If the dataset is state-only, convert object positions, robot/end-effector positions, joint states, and goal states into compact text tokens. This is not a strict reproduction of KAT, but it is acceptable if clearly labeled as a state-token KAT variant.

2. Action side: You must design action tokens appropriate to the environment. Encode end-effector trajectories into a token sequence that can be decoded back into valid robot actions.

The original paper found that the number of keypoints and action tokens matters; it reports good performance around 10 - 20 keypoints and around 20 action tokens in its own setting. Therefore test at least two token budgets, such as 10 vs. 20 observation tokens or 10 vs. 20 action tokens.

3. At least one successful decode-and-replay sanity check must be shown before proceeding.

Step 4: In-context KAT implementation

1. Construct prompts from demonstration token pairs and a held-out input observation, then generate the output action token sequence.
2. The parser from generated tokens back to executable actions must be robust and should log invalid predictions.

Running a local LLM is encouraged. Ollama is the default recommended interface because it provides a simple local model server and command-line workflow; llama.cpp, LM Studio, and vLLM are also good alternatives.

Step 5: Evaluation, ablation and reproducibility

1. Evaluate on fixed seeds and report success rates separately for seen and shifted / unseen conditions.
2. Require at least one meaningful ablation. Good choices are the number of demos, the number of keypoints K, or the number of action tokens.
3. Require a clean README, environment file, data manifest, evaluation script, and a results CSV.
4. The project should be reproducible from one command or one short script sequence.

Step 6: (OPTIONAL) Few-shot fine-tuning

Fine-tune the local LLM on tokenized demonstration data. Recommended approach:

- Use Hugging Face Transformers + TRL SFTTrainer.
- Use PEFT LoRA or QLoRA.
- Train only adapters, not the full model.
- Keep the base model frozen.

- Use fixed precision and stable token formatting.
- Use small batches and gradient accumulation.

PEFT/QLoRA is recommended because it reduces memory use by training adapters on top of a quantized model; Hugging Face's PEFT documentation specifically describes combining quantization with PEFT, and QLoRA-style 4-bit fine-tuning is a common way to fit larger models on limited GPU memory.

Notes:

*You may discuss the general concepts in this project with other students, but you must implement the program on your own. **No sharing of code or reports is allowed.** Violation of this policy can result in a grade penalty.*

Late submission is not acceptable for this project.